Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position
Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review



0

Notifications

You have no notifications
See all
Priyadharshini C 251801247-2025-AIDS P2
Profile Grades Calendar Private files Reports
Preferences
Log out
P2

- Home
- Dashboard
- My courses
- More

Search courses Search courses
Open course index
Open block drawer

1. CS23331-DAA-2025-CSE
2. 1-Number of Zeros in a Given Array

1-Number of Zeros in a Given Array

Started on	Thursday, 27 August 2026, 10:37 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:38 AM
Time taken	1 min 2 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer:(penalty regime: 0 %)

```c
#include <stdio.h>

int count(int a[], int l, int r)
{
    if (l > r)
        return 0;

    if (a[l] == 0)
        return r - l + 1;

    int mid = (l + r) / 2;

    if (a[mid] == 1)
        return count(a, mid + 1, r);
    else
        return count(a, l, mid - 1) + (r - mid + 1);
}

int main()
{
    int n, i;
    scanf("%d", &n);

    int a[n];

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("%d", count(a, 0, n - 1));

    return 0;
}
```


```c
#include <stdio.h>

int count(int a[], int l, int r)
{
    if (l > r)
        return 0;

    if (a[l] == 0)
        return r - l + 1;
```

```
    int mid = (l + r) / 2;

    if (a[mid] == 1)
        return count(a, mid + 1, r);
    else
        return count(a, l, mid - 1) + (r - mid + 1);
}

int main()
{
    int n, i;
    scanf("%d", &n);

    int a[n];

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("%d", count(a, 0, n - 1));

    return 0;
}
```

Feedback

Input Expected Got

```
5
1
1
1
0
0
```

2 2

```
10
1
1
1
1
1
1
1
1
1
1
```

0 0

```
8
0
0
0
0
0
0
0
0
```

8 8

Input Expected Got

```
17
1
1
1
1
1
1
1
1
1
1
1
1
1
1
1
0
0
```

```
2 2
```

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course

Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position
Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review

2-Majority Element

Started on	Thursday, 27 August 2026, 10:38 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:39 AM
Time taken	33 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer:(penalty regime: 0 %)

```
#include <stdio.h>

int main()
{
    int n, i;
    int a[50000];

    scanf("%d", &n);

    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    int candidate = a[0];
    int count = 1;

    for(i = 1; i < n; i++)
    {
        if(a[i] == candidate)
            count++;
        else
            count--;

        if(count == 0)
        {
            candidate = a[i];
            count = 1;
```

```
        }
    }

    printf("%d", candidate);

    return 0;
}
```

```c
#include <stdio.h>

int main()
{
    int n, i;
    int a[50000];

    scanf("%d", &n);

    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    int candidate = a[0];
    int count = 1;

    for(i = 1; i < n; i++)
    {
        if(a[i] == candidate)
            count++;
        else
            count--;

        if(count == 0)
        {
            candidate = a[i];
            count = 1;
        }
    }

    printf("%d", candidate);

    return 0;
}
```

Feedback

Input Expected Got

```
3
3 2 3 3        3
```

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course

You are logged in as Priyadharshini C 251801247-2025-AIDS (Log out)
Data retention summary

Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position
Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review



0

Notifications

You have no notifications
See all
Priyadharshini C 251801247-2025-AIDS P2
Profile Grades Calendar Private files Reports
Preferences
Log out
P2

- Home
- Dashboard
- My courses
- More

Search courses Search courses
Open course index
Open block drawer

1. CS23331-DAA-2025-CSE
2. 3-Finding Floor Value

3-Finding Floor Value

Started on	Thursday, 27 August 2026, 10:39 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:40 AM
Time taken	1 min 21 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Problem Statement:
Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array
Next n lines Contains n numbers – Elements of an array
Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer:(penalty regime: 0 %)

```c
#include <stdio.h>

int findFloor(int arr[], int n, int x) {
    int low = 0, high = n - 1;
    int floorVal = -1; // Default if no element <= x exists

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == x) {
            return arr[mid]; // Exact match is the floor
        } else if (arr[mid] < x) {
            floorVal = arr[mid]; // Potential floor, try searching the right half for a closer one
            low = mid + 1;
        } else {
            high = mid - 1; // arr[mid] > x, so search the left half
        }
    }
    return floorVal;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    scanf("%d", &x);

    int result = findFloor(arr, n, x);
    printf("%d\n", result);

    return 0;
}
```

```c
#include <stdio.h>

int findFloor(int arr[], int n, int x) {
    int low = 0, high = n - 1;
```

```c
    int floorVal = -1; // Default if no element <= x exists

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == x) {
            return arr[mid]; // Exact match is the floor
        } else if (arr[mid] < x) {
            floorVal = arr[mid]; // Potential floor, try searching the right half for a closer one
            low = mid + 1;
        } else {
            high = mid - 1; // arr[mid] > x, so search the left half
        }
    }
    return floorVal;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int x;
    scanf("%d", &x);

    int result = findFloor(arr, n, x);
    printf("%d\n", result);

    return 0;
}
```

Feedback

Input Expected Got

```
6
1
2
8
10    2        2
12
19
5


5
10
22
85    85       85
108
129
100
```

Input	Expected	Got
7 3 5 7 9 11 13 15 10	9 9	9

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course

Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position
Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review



0

Notifications

You have no notifications
See all
Priyadharshini C 251801247-2025-AIDS P2
Profile Grades Calendar Private files Reports
Preferences
Log out
P2

- Home
- Dashboard
- My courses
- More

Search courses Search courses
Open course index
Open block drawer

1. CS23331-DAA-2025-CSE
2. 4-Two Elements sum to x

4-Two Elements sum to x

Started on	Thursday, 27 August 2026, 10:41 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:42 AM
Time taken	55 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Problem Statement:
Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as “No”.
Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array
Next n lines Contains n numbers – Elements of an array
Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1
Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value “x”)

Answer:(penalty regime: 0 %)

```c
#include <stdio.h>

// Recursive binary search (Divide and Conquer approach)
int search(int a[], int low, int high, int key) {
    if (low > high) return -1;

    int mid = low + (high - low) / 2;
    if (a[mid] == key) return mid;
    if (a[mid] > key) return search(a, low, mid - 1, key);
    return search(a, mid + 1, high, key);
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    int x;
    scanf("%d", &x);

    // Check every element and search its complement using recursion
    for (int i = 0; i < n; i++) {
        int rem = x - a[i];
        int j = search(a, i + 1, n - 1, rem);

        if (j != -1) {
            printf("%d\n%d\n", a[i], a[j]);
            return 0;
        }
    }

    printf("No\n");
    return 0;
}
```

```c
#include <stdio.h>

// Recursive binary search (Divide and Conquer approach)
int search(int a[], int low, int high, int key) {
```

```
    if (low > high) return -1;

    int mid = low + (high - low) / 2;
    if (a[mid] == key) return mid;
    if (a[mid] > key) return search(a, low, mid - 1, key);
    return search(a, mid + 1, high, key);
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    int x;
    scanf("%d", &x);

    // Check every element and search its complement using recursion
    for (int i = 0; i < n; i++) {
        int rem = x - a[i];
        int j = search(a, i + 1, n - 1, rem);

        if (j != -1) {
            printf("%d\n%d\n", a[i], a[j]);
            return 0;
        }
    }

    printf("No\n");
    return 0;
}
```

Feedback

Input Expected Got

```
4
2
4     4         4
8     10        10
10
14


5
2
4
6     No        No
8
10
100
```

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course

You are logged in as Priyadharshini C 251801247-2025-AIDS (Log out)
Data retention summary

Skip to main content

Expand Collapse General Highlighted

Expand Collapse Finding Time Complexity of Algorithms Highlighted

- Problem 1: Finding Complexity using Counter Method
- Problem 2: Finding Complexity using Counter method
- Problem 3: Finding Complexity using Counter Method
- Problem 4: Finding Complexity using Counter Method
- Problem 5: Finding Complexity using counter method

Expand Collapse Greedy Algorithms Highlighted

- 1-G-Coin Problem
- 2-G-Cookies Problem
- 3-G-Burger Problem
- 4-G-Array Sum max problem
- 5-G-Product of Array elements-Minimum

Expand Collapse Divide and Conquer Highlighted

- 1-Number of Zeros in a Given Array
- 2-Majority Element
- 3-Finding Floor Value
- 4-Two Elements sum to x
- 5-Implementation of Quick Sort

Expand Collapse Dynamic Programming Highlighted

- 1-DP-Playing with Numbers
- 2-DP-Playing with chessboard
- 3-DP-Longest Common Subsequence
- 4-DP-Longest non-decreasing Subsequence

Expand Collapse Competitive Programming Highlighted

- 1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity
- 2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity
- 3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity
- 4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity
- 5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity
- 6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Side Block Position
Skip Quiz navigation

Quiz navigation

Question 1 This page
Finish review



0

Notifications

You have no notifications
See all
Priyadharshini C 251801247-2025-AIDS P2
Profile Grades Calendar Private files Reports
Preferences
Log out
P2

- Home
- Dashboard
- My courses
- More

Search courses Search courses
Open course index
Open block drawer

1. CS23331-DAA-2025-CSE
2. 5-Implementation of Quick Sort

5-Implementation of Quick Sort

Started on	Thursday, 27 August 2026, 10:42 AM
State	Finished
Completed on	Thursday, 27 August 2026, 10:42 AM
Time taken	39 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct
Mark 1.00 out of 1.00
Flag question

Question text

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void quickSort(int a[], int low, int high) {
    if (low >= high) return;

    int pivot = a[high];
    int i = low;

    // Partition elements around the pivot
    for (int j = low; j < high; j++) {
        if (a[j] < pivot) {
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            i++;
        }
    }
    // Place pivot in its correct position
    int temp = a[i];
    a[i] = a[high];
    a[high] = temp;

    // Recursively sort left and right sub-arrays
    quickSort(a, low, i - 1);
    quickSort(a, i + 1, high);
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    return 0;
}
```

```c
#include <stdio.h>

void quickSort(int a[], int low, int high) {
    if (low >= high) return;

    int pivot = a[high];
    int i = low;

    // Partition elements around the pivot
    for (int j = low; j < high; j++) {
        if (a[j] < pivot) {
            int temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            i++;
        }
    }
    // Place pivot in its correct position
    int temp = a[i];
    a[i] = a[high];
    a[high] = temp;

    // Recursively sort left and right sub-arrays
    quickSort(a, low, i - 1);
    quickSort(a, i + 1, high);
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    printf("\n");

    return 0;
}
```

Feedback

Input	Expected	Got
5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98
10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114
12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90

Passed all tests!

Correct
Marks for this submission: 1.00/1.00.
Finish review

Back to Course

^
Footer (Left)
Footer (Middle)
Footer (Middle 2)
Footer (Right)
You are logged in as Priyadharshini C 251801247-2025-AIDS (Log out)
Data retention summary